



Documentation of completed tasks

2025/2026

High Availability Distributed Systems

Task 2

Field of study: Computer Science

Team members:

Adam Chabraszewski 193373

Mateusz Fydrych 193410

Jakub Jędrzejczyk 188752

Gdańsk, 2025/2026

Contents

1	Introduction	2
1.1	Roles in the implementation of tasks	2
1.2	The purpose of the task	2
2	Assumptions for the implementation of the task	3
2.1	Technical and non-technical assumptions	3
2.2	Technology stack	3
2.3	Expected results of the task implementation	3
3	Implementation of the task	4
3.1	Docker Desktop and the getting-started container	4
3.2	Our application	7
3.3	Updating our application	10
3.4	Sharing our app	12
3.5	Persisting our database	13
3.6	Using bind mounts	15
3.7	Multi-container apps	16
3.8	Using Docker Compose	19
3.9	Finishing the project	21
4	Conclusions	25

1 Introduction

1.1 Roles in the implementation of tasks

The tasks were completed collaboratively by all team members. We worked together during the configuration, testing, and documentation stages. Each team member participated in analyzing the instructions, running Docker commands, solving encountered issues, and preparing screenshots for the report.

1.2 The purpose of the task

The purpose of the task was to become familiar with Docker Desktop, container images, containers, port mapping, Dockerfiles, and the process of building, running, updating, and sharing a containerized application. The task was based on the official Docker getting started tutorial.

2 Assumptions for the implementation of the task

We worked synchronously using Discord and discussed the tasks in real time. This allowed us to learn together and detect possible issues quickly. The task was completed on macOS.

2.1 Technical and non-technical assumptions

The main technical assumption was that Docker Desktop was installed and working correctly on the local machine. We also assumed that the terminal, web browser, and IDE were available. From a non-technical point of view, we assumed active cooperation between team members and regular communication during the implementation.

2.2 Technology stack

The following tools and technologies were used during the task:

- Docker Desktop,
- Docker Hub,
- terminal,
- web browser,
- PyCharm IDE,
- Node.js application provided in the tutorial,
- macOS operating system.

2.3 Expected results of the task implementation

The expected result was to successfully run a Docker container, build a custom Docker image using a Dockerfile, run a simple todo application, update the application, rebuild the image, push it to Docker Hub, and understand why data can be lost after removing a container.

At the end of the task, we expected to have a working todo application that could be run in different ways: as a single container, with persistent storage, with a separate database container, and as a Docker Compose project.

3 Implementation of the task

The implementation was divided into several parts. We started with the simplest Docker commands and then moved step by step to more advanced topics, such as volumes, bind mounts, container networking, and Docker Compose. This structure helped us understand how each new concept solves a specific problem from the previous stage.

3.1 Docker Desktop and the getting-started container

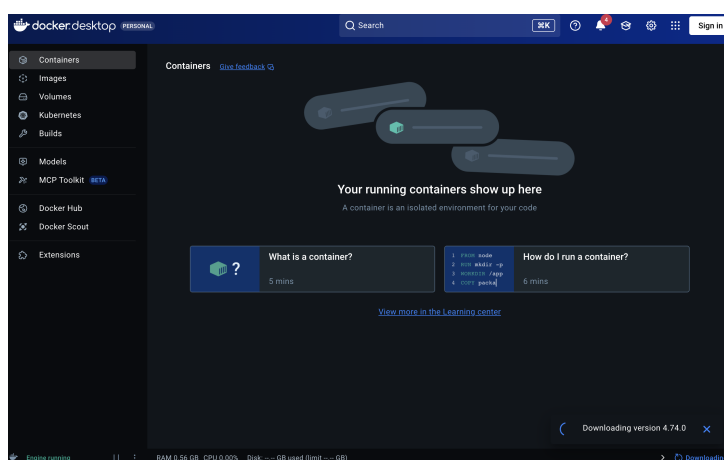


Figure 1: Docker Desktop available on the laptop

The first task from the Docker tutorial available at <https://www.docker.com/101-tutorial/> was to download Docker Desktop. However, Docker Desktop was already installed on our machine. We only noticed that an update was required.

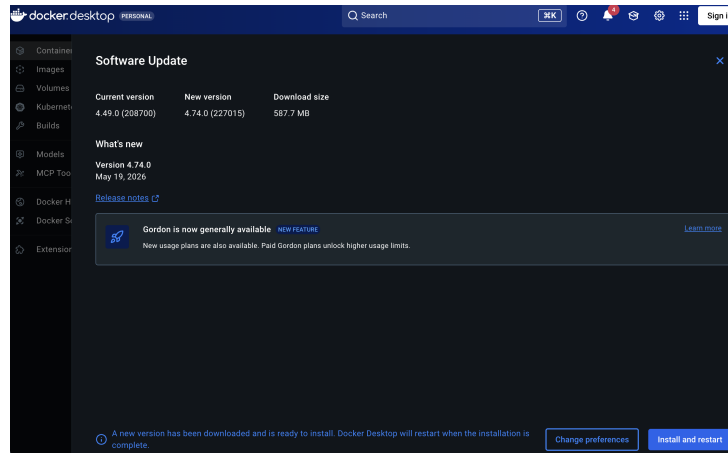


Figure 2: Comparison between the previous and the new version

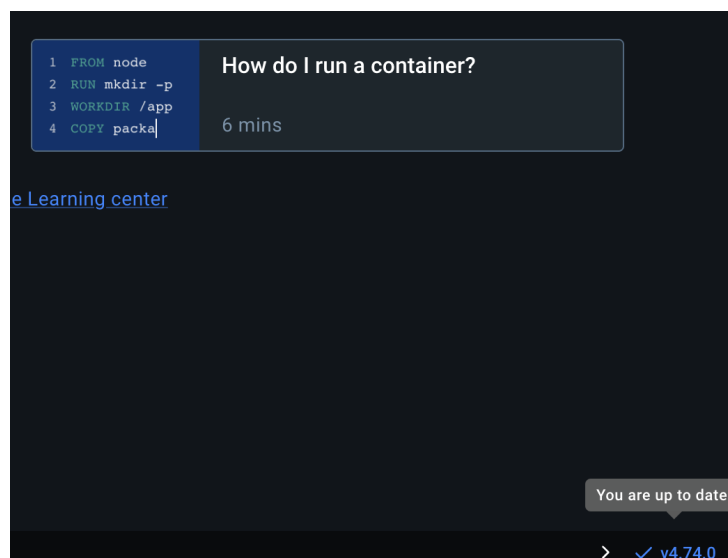


Figure 3: Docker Desktop after updating to the latest version

After the update, we were ready to continue with the next steps. According to the instructions, we had to open Docker Desktop and run the following command in the terminal:

```
docker run -dp 80:80 docker/getting-started
```

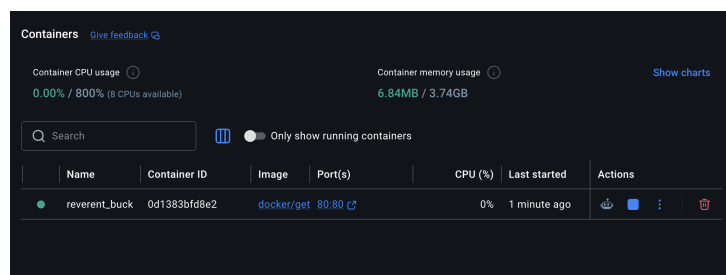
```

mat@MacBook-Air-3 hads % docker run -dp 80:80 docker/getting-started
Unable to find image 'docker/getting-started:latest' locally
latest: Pulling from docker/getting-started
2f61404bb4b8: Pull complete
14f901bbf056: Pull complete
fa3f58a317be: Pull complete
476bb2a1cc22: Pull complete
33a28b928e89: Pull complete
a879581b8e12: Pull complete
d0193f05f10f: Pull complete
261da4162673: Pull complete
a60aada4c44a: Pull complete
Digest: sha256:d79336f4812b6547a53e735480dde67f8f7071b414fbd9297609ffb989abc1
Status: Downloaded newer image for docker/getting-started:latest
0d1383bfd8e25a115a2cd288db367b4e6f6a46064297778433081363eca93f9f
mat@MacBook-Air-3 hads %

```

Figure 4: Successful execution of the docker run command

After running the container in detached mode with port mapping 80:80, we were able to access the application from the browser. From the lectures, we knew that the value on the left side represents the host port, while the value on the right side represents the container port.



The screenshot shows the Docker Desktop interface. At the top, it displays 'Containers' with a 'Give feedback' link. Below this, there are two status indicators: 'Container CPU usage' at 0.00% / 800% (8 CPUs available) and 'Container memory usage' at 6.84MB / 3.74GB. A 'Show charts' link is also present. A search bar and a toggle for 'Only show running containers' are visible. The main part of the interface is a table listing the running containers.

Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
reverent_buck	0d1383bfd8e2	docker/get	80:80	0%	1 minute ago	[Stop] [Restart] [Refresh] [Delete]

Figure 5: Running container visible in Docker Desktop

Then, we opened the browser and entered:

`http://localhost`

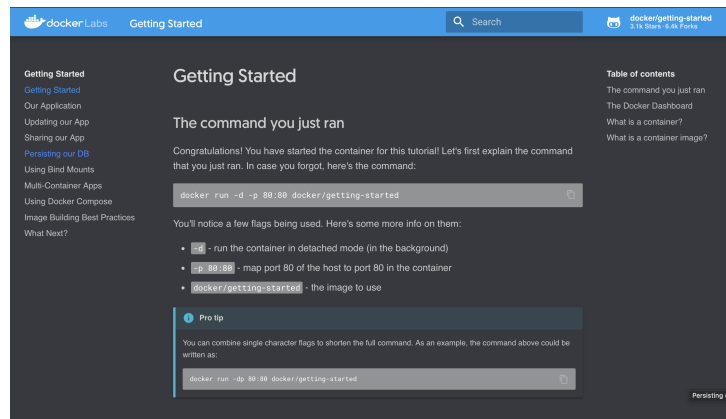


Figure 6: Localhost view

After entering the localhost address, we could see a user-friendly page with the next tasks listed in the left panel. The main page contained descriptions of the tasks to complete.

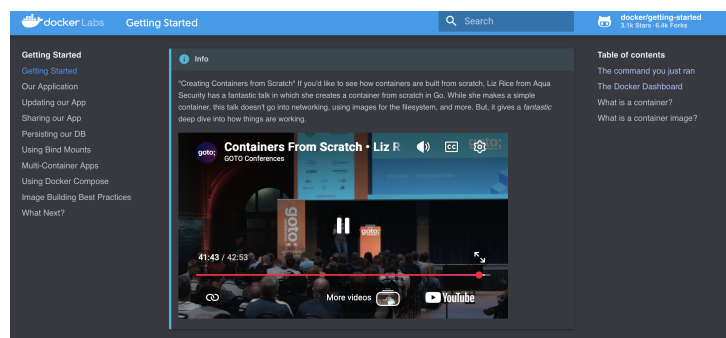


Figure 7: Docker tutorial page opened locally

The first part of the tutorial, “Getting Started”, was mainly a theoretical introduction. We also watched the suggested video about containers, which was a useful reminder of the knowledge presented during lectures.

3.2 Our application

The next step was to run the todo application from the tutorial.

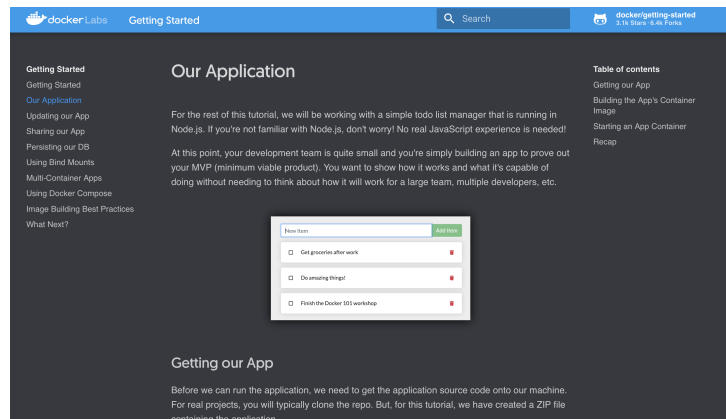


Figure 8: Todo application – localhost view

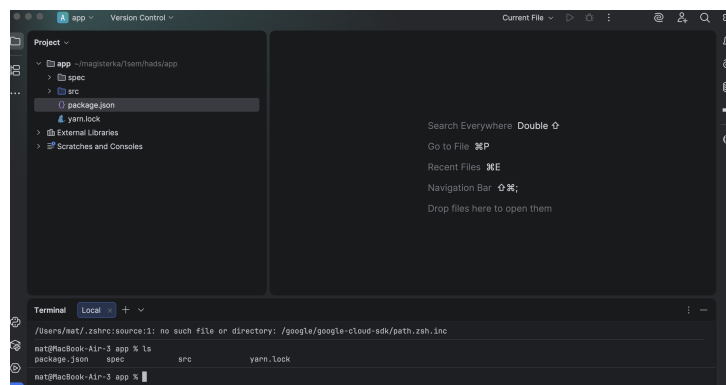


Figure 9: Prepared application structure

The suggested IDE was Visual Studio Code. However, we decided to use PyCharm from JetBrains. Even though PyCharm is mainly used for Python-related projects, it was sufficient for this task because it provides syntax highlighting, an easy-to-use interface, and an integrated terminal.

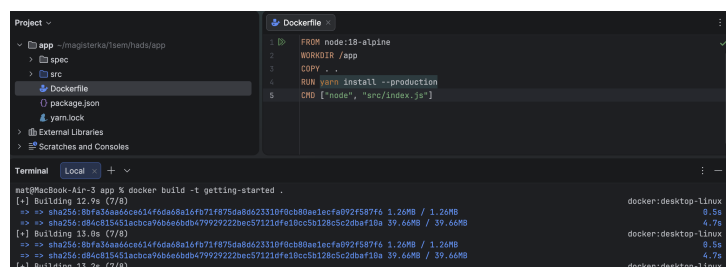


Figure 10: Successfully added Dockerfile

Following the instructions, we added a simple Dockerfile for our todo application:

```
FROM node:18-alpine
WORKDIR /app
COPY . .
RUN yarn install --production
CMD ["node", "src/index.js"]
```

It was important to make sure that the file was named exactly **Dockerfile**, without any additional extension such as **.txt**. Some editors may automatically append such extensions, which would cause an error in the next step.

Then, we opened the terminal in the application directory and built the container image using the following command:

```
docker build -t getting-started .
```

This command used the Dockerfile to build a new container image. During the process, Docker downloaded the required layers because the image was based on **node:18-alpine**. After that, the application files were copied into the image and the dependencies were installed using **yarn**. The **CMD** instruction defined the default command used to start the application inside the container.

The **-t** flag assigned a human-readable name to the image. In this case, the image was named **getting-started**. The dot at the end of the command indicated that Docker should look for the Dockerfile in the current directory.

After building the image, we ran the application and opened it in the browser.

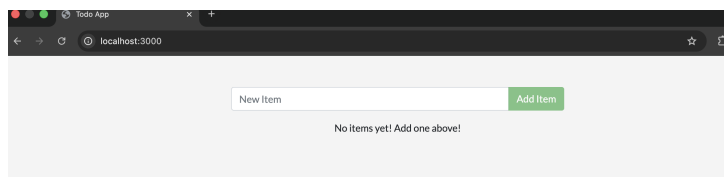


Figure 11: Successfully running application

After building and running the application, we were able to see the result at **localhost:3000**.

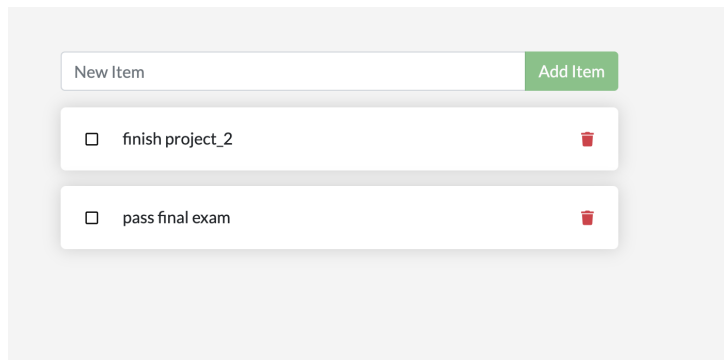


Figure 12: Successfully added new todo items

As suggested in the tutorial, we added two example todo items to the list. Even though the user interface and functionality were very basic, the application worked as expected and could be further extended.

3.3 Updating our application

The next step was to update the application. We modified the code in the following file:

`src/static/js/app.js`

The text displayed when there were no todo items was changed from:

```
<p className="text-center">No items yet! Add one above!</p>
```

to:

```
<p className="text-center">You have no todo items yet! Add one above!</p>
```

After saving the change, we rebuilt the image. Then, we tried to run the updated image using the command:

```
docker run -dp 3000:3000 getting-started
```

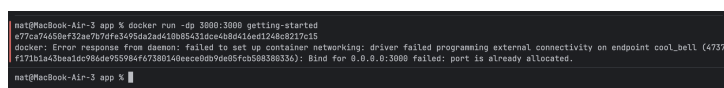


Figure 13: Error in the terminal while launching the docker run command

We received the error visible in the figure. This error was expected because the previous container was still running and using the same port.

```
nat@MacBook-Air-3 app % docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
f5ff0de0bb65   ac384b7de132   "docker-entrypoint.s..." 19 hours ago   Up 19 hours   0.0.0.0:3000->3000/tcp, :::3000->3000/tcp   modest_night
0d1383bfd9d2   docker/getting-started   "/docker-entrypoint..." 25 hours ago   Up 25 hours   0.0.0.0:80->80/tcp, :::80->80/tcp   reverent_buck
nat@MacBook-Air-3 app % docker stop f5ff0de0bb65
f5ff0de0bb65

nat@MacBook-Air-3 app % docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
0d1383bfd9d2   docker/getting-started   "/docker-entrypoint..." 25 hours ago   Up 25 hours   0.0.0.0:80->80/tcp, :::80->80/tcp   reverent_buck
nat@MacBook-Air-3 app % docker rm f5ff0de0bb65
f5ff0de0bb65

nat@MacBook-Air-3 app %
```

Figure 14: Process of stopping and removing the container

The figure shows that we successfully stopped the container and removed it from the system. A similar result could also be achieved with the following command:

```
docker rm -f <container_id>
```

Alternatively, the same operation could be done using the Docker Desktop user interface.

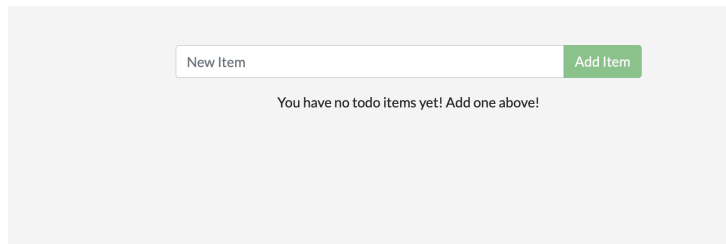


Figure 15: Successfully running the updated application

After running the container again, we could see that the updated application was visible in the browser. It is important to note that the previous todo items disappeared. This happened because the old container was removed, and the data stored inside it was not persisted. This is expected behavior when no volume is used.

3.4 Sharing our app

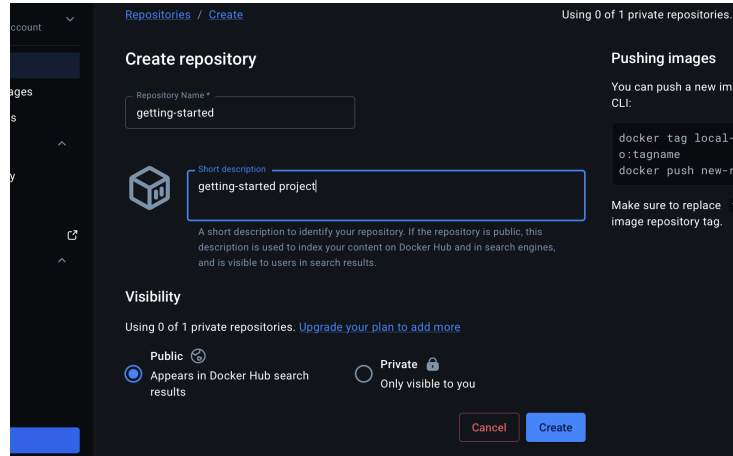


Figure 16: Creating a new repository in Docker Hub

The next part of the task was to share our application image using Docker Hub. First, we created a new repository in Docker Hub.

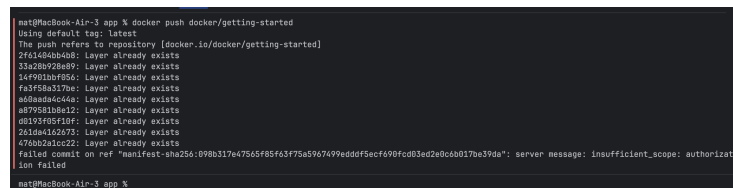


Figure 17: Expected error while pushing the image

At first, pushing the image failed because Docker was looking for an image with a name that did not exist locally. To solve this problem, we used the `docker tag` command:

```
docker tag getting-started YOUR-USER-NAME/getting-started
```

This command created a new tag for the existing local image, making it compatible with the Docker Hub repository name.

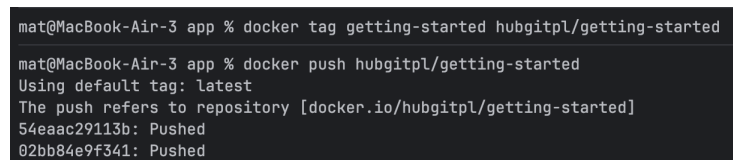


Figure 18: Tagging the image and pushing it to Docker Hub

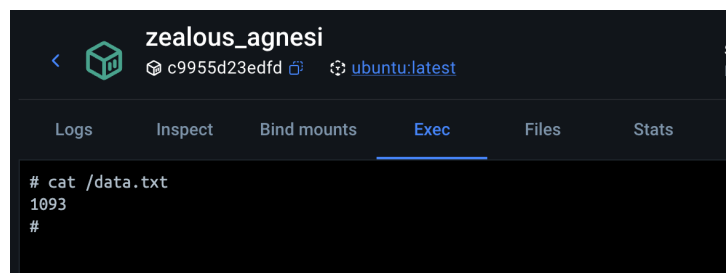
After tagging the image correctly, we pushed it to Docker Hub. Then, the image could be easily downloaded and run on another laptop.

3.5 Persisting our database

The first step was to run a simple Ubuntu container that generated a random number from 1 to 10000.

```
mat@MacBook-Air-3 app % docker run -d ubuntu bash -c "shuf -i 1-10000 -n 1 -o /data.txt && tail -f /dev/null"
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
2113f8d7eb32: Pull complete
4a7720858461: Pull complete
a7091e06226b: Download complete
Digest: sha256:f3d28607ddd78734bb7f71f117f3c6706c666b8b76cbff7c9ff6e5718d46ff64
Status: Downloaded newer image for ubuntu:latest
c9955d23edfd391141e786df55b8438b7dbbe24db811780819d19af9a3374dea
```

Figure 19: Docker command executed successfully



```
# cat /data.txt
1093
#
```

Figure 20: Validating the output by using exec in Docker Desktop

```
mat@MacBook-Air-3 app % docker volume create todo-db
todo-db

mat@MacBook-Air-3 app % docker rm -f e599
e599

mat@MacBook-Air-3 app % docker run -dp 3000:3000 -v todo-db:/etc/todos getting-started
396945c1690ce3bd2417923266960e467958e022ea340fcad54061e4b7ed25f

mat@MacBook-Air-3 app %
```

Figure 21: Docker commands in the terminal

In the terminal, we first created a new volume. Then, we removed the old container and started a new container with the volume mounted:

```
docker volume create todo-db
docker rm -f <container_id>
docker run -dp 3000:3000 -v todo-db:/etc/todos getting-started
```

After opening the application, we added three todo tasks. Then, even after removing the container and starting it again with the same command, the data was still available:

```
docker run -dp 3000:3000 -v todo-db:/etc/todos getting-started
```

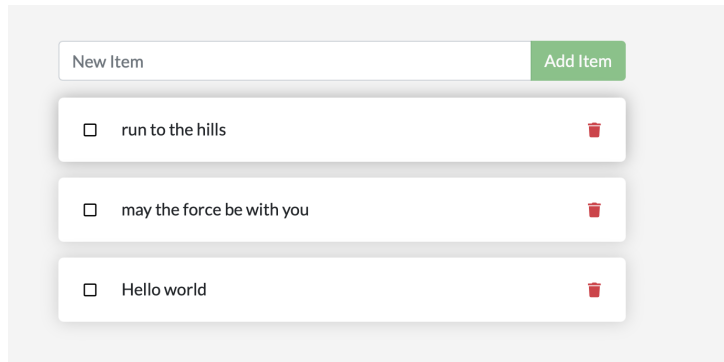


Figure 22: The todo tasks are still visible after removing and running the container again

Using the following command, we were able to check where the data was stored:

```
mat@MacBook-Air-3 app % docker volume inspect todo-db
[
  {
    "CreatedAt": "2026-05-23T18:05:53Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/todo-db/_data",
    "Name": "todo-db",
    "Options": null,
    "Scope": "local"
  }
]
```

In this part of the task, we analyzed why the application data disappeared after removing and recreating the container. The reason was that the application stored data inside the container filesystem. When the container was removed, this data was removed as well.

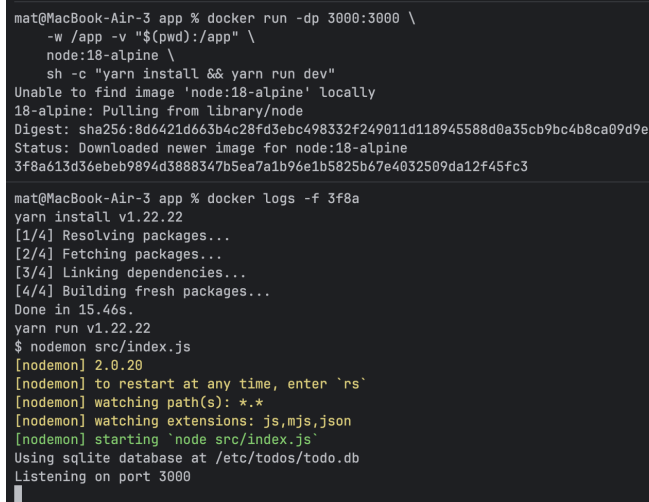
To avoid this problem, Docker volumes can be used. A volume allows data to be stored outside the container lifecycle. As a result, even if the container is removed and recreated, the application data can still be available.

3.6 Using bind mounts

After understanding the difference between volumes and bind mounts, we started the practical part. First, we ran the following command:

```
docker run -dp 3000:3000 \
  -w /app -v "$(pwd):/app" \
  node:18-alpine \
  sh -c "yarn install && yarn run dev"
```

This command started the application as usual, but this time with a bind mount. It is useful to note that Docker requires absolute paths for bind mounts. In this case, using `$(pwd)` was a convenient way to use the current directory without typing the full absolute path manually.



```
mat@MacBook-Air-3 app % docker run -dp 3000:3000 \
  -w /app -v "$(pwd):/app" \
  node:18-alpine \
  sh -c "yarn install && yarn run dev"
Unable to find image 'node:18-alpine' locally
18-alpine: Pulling from library/node
Digest: sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
Status: Downloaded newer image for node:18-alpine
3f8a613d36eb9894d3888347b5ea7a1b96e1b5825b67e4032509da12f45fc3

mat@MacBook-Air-3 app % docker logs -f 3f8a
yarn install v1.22.22
[1/4] Resolving packages...
[2/4] Fetching packages...
[3/4] Linking dependencies...
[4/4] Building fresh packages...
Done in 15.46s.
yarn run v1.22.22
$ nodemon src/index.js
[nodemon] 2.0.20
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting 'node src/index.js'
Using sqlite database at /etc/todos/todo.db
Listening on port 3000
```

Figure 23: Commands used to run the container and check its logs

Then, we changed line 109 in the application:

```
- {submitting ? 'Adding...' : 'Add Item'}
+ {submitting ? 'Adding...' : 'Add'}
```

After saving the file, the application automatically reflected the newest change. This showed that bind mounts are useful during development because changes in local files can be visible inside the container without rebuilding the image each time.

3.7 Multi-container apps

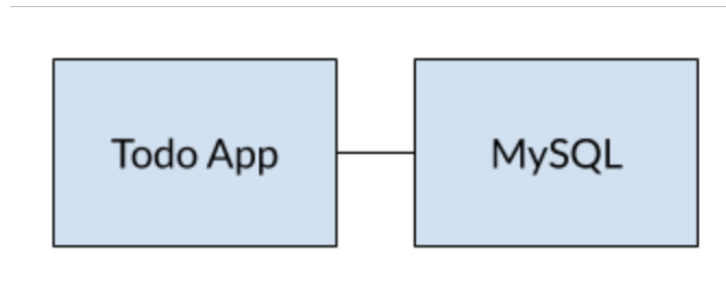


Figure 24: Target architecture in our containerized app

While setting up containers in the same network, we followed a simple rule: if two containers are on the same Docker network, they can communicate with each other. If they are not on the same network, they cannot communicate directly.

```
mat@MacBook-Air-3 app % docker network create todo-app
15bd435510b8a00679b037bc0e346e79d0f0658611b4dd80f222b4d361695641

mat@MacBook-Air-3 app % docker run -d \
--network todo-app --network-alias mysql \
-v todo-mysql-data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=secret \
-e MYSQL_DATABASE=todos \
mysql:8.0
mysql:8.0
Unable to find image 'mysql:8.0' locally
8.0: Pulling from library/mysql
4fcb28d1dfcf: Pull complete
ad770837e789: Pull complete
70a53223cb4c: Pull complete
7b9f24d93d52: Pull complete
11c0b299cb47: Pull complete
7ce425b17caf: Pull complete
e9b2852d2a9d: Pull complete
605b7f42ac6e: Pull complete
1959e4a06a1c: Pull complete
42505e6e6b91: Pull complete
fa30ab141b5a: Pull complete
7a0e30178db8: Download complete
578f543767f0: Download complete
Digest: sha256:7dcddc01f13bab2f15cde676d44d01f61fc9f99fe7785e86196dfc07d358ae2b
Status: Downloaded newer image for mysql:8.0
891b90fa1a8938665cdbec3865024c1c1802bf4524931efa7ab49d817767966
mat@MacBook-Air-3 app %
```

Figure 25: Creating the network and running the container within this network

Docker recognized that we wanted to use a named volume and created one automatically for us.

```

mat@MacBook-Air-3 app % docker exec -it 891b90 mysql -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES
->
-> ;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
| todos |
+-----+
5 rows in set (0.03 sec)

mysql>

```

Figure 26: Rows in the SQL database

After MySQL was up and running, we used the `nicolaka/netshoot` container. This container includes many tools that are useful for troubleshooting and debugging networking issues.

The IP address was resolved correctly:

;; ANSWER SECTION:

```
mysql.          600      IN      A        172.18.0.2
```

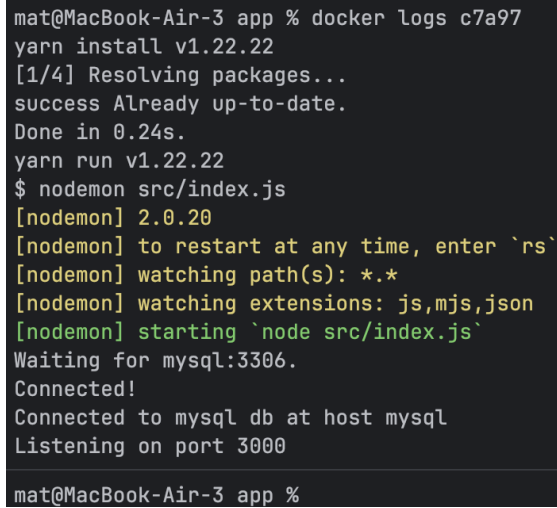
The `todo` application supports a few environment variables used to configure the database connection:

- `MYSQL_HOST` – the hostname of the running MySQL server,
- `MYSQL_USER` – the username used for the connection,
- `MYSQL_PASSWORD` – the password used for the connection,
- `MYSQL_DB` – the database used after connecting.

Using environment variables for connection settings is acceptable for development, but it is not recommended as the only security solution in production.

The command used to run the application with the MySQL database was:

```
docker run -dp 3000:3000 \  
  -w /app -v "$(pwd):/app" \  
  --network todo-app \  
  -e MYSQL_HOST=mysql \  
  -e MYSQL_USER=root \  
  -e MYSQL_PASSWORD=secret \  
  -e MYSQL_DB=todos \  
  node:18-alpine \  
  sh -c "yarn install && yarn run dev"
```

A terminal window with a dark background and light-colored text. The text shows the execution of 'docker logs' for a container named 'c7a97'. It displays the output of 'yarn install' and 'yarn run', which includes 'nodemon' starting and connecting to a MySQL database. The prompt 'mat@MacBook-Air-3 app %' is visible at the top and bottom of the terminal output.

```
mat@MacBook-Air-3 app % docker logs c7a97  
yarn install v1.22.22  
[1/4] Resolving packages...  
success Already up-to-date.  
Done in 0.24s.  
yarn run v1.22.22  
$ nodemon src/index.js  
[nodemon] 2.0.20  
[nodemon] to restart at any time, enter `rs`  
[nodemon] watching path(s): *.*  
[nodemon] watching extensions: js,mjs,json  
[nodemon] starting `node src/index.js`  
Waiting for mysql:3306.  
Connected!  
Connected to mysql db at host mysql  
Listening on port 3000  
mat@MacBook-Air-3 app %
```

Figure 27: Container logs result

In the picture, we can see confirmation that the container is using the MySQL database.

```

mat@MacBook-Air-3 app % docker exec -it 891b90fa1a89 mysql -p todos
Enter password:
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 12
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> select * from todo_items;
+-----+-----+-----+
| id                | name | completed |
+-----+-----+-----+
| 2dfea33f-69ba-4890-833c-f58bbb236fb6 | todo 1 | 0 |
| be5d1217-6e21-43e5-9221-6e8d15cf29fb | todo 2 | 0 |
| 695175f9-20b9-4165-9da0-4401b55aed5b | todo 3 | 0 |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>

```

Figure 28: Data in the database

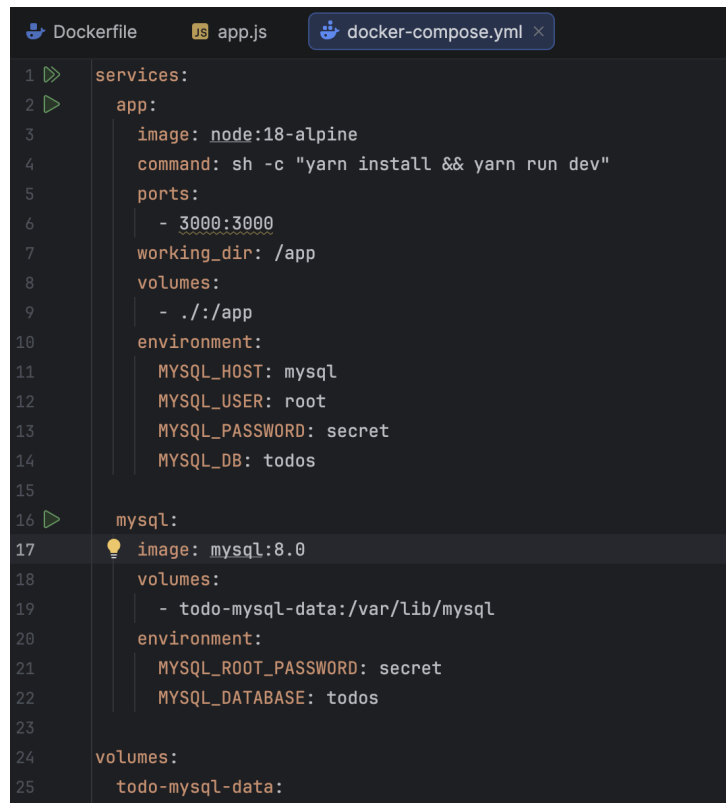
Here, we can see how and where the data from our application is stored.

At this point, the application stored its data in an external database running in a separate container. We also learned how basic container networking works and how service discovery can be performed using DNS.

However, starting the whole application manually required many steps. We had to create a network, start containers, specify environment variables, expose ports, and remember the correct commands. This is why Docker Compose is useful.

3.8 Using Docker Compose

Docker Compose is a tool that helps define and share multi-container applications. With Compose, we can create a YAML file that defines all required services. Then, with a single command, we can start or stop the whole application.

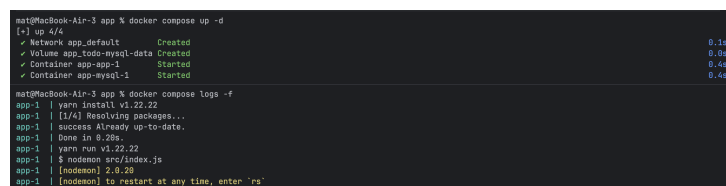


```

1  services:
2    app:
3      image: node:18-alpine
4      command: sh -c "yarn install && yarn run dev"
5      ports:
6        - 3000:3000
7      working_dir: /app
8      volumes:
9        - ./:/app
10     environment:
11       MYSQL_HOST: mysql
12       MYSQL_USER: root
13       MYSQL_PASSWORD: secret
14       MYSQL_DB: todos
15
16   mysql:
17     image: mysql:8.0
18     volumes:
19       - todo-mysql-data:/var/lib/mysql
20     environment:
21       MYSQL_ROOT_PASSWORD: secret
22       MYSQL_DATABASE: todos
23
24   volumes:
25     todo-mysql-data:

```

Figure 29: Finished Docker Compose file



```

mat@MacBook-Air-3 app % docker compose up -d
[+] up 4/4
✓ Network app_default Created 0.1s
✓ Volume app_todo-mysql-data Created 0.0s
✓ Container app-app-1 Started 0.4s
✓ Container app-mysql-1 Started 0.4s

mat@MacBook-Air-3 app % docker compose logs -f
app-1 | yarn install v1.22.22
app-1 | [1/4] Resolving packages...
app-1 | success Already up-to-date.
app-1 | Done in 0.28s.
app-1 | yarn run v1.22.22
app-1 | $ node src/index.js
app-1 | [nodemon] 2.0.28
app-1 | [nodemon] to restart at any time, enter `rs`

```

Figure 30: Docker commands used to start Docker Compose and check live logs

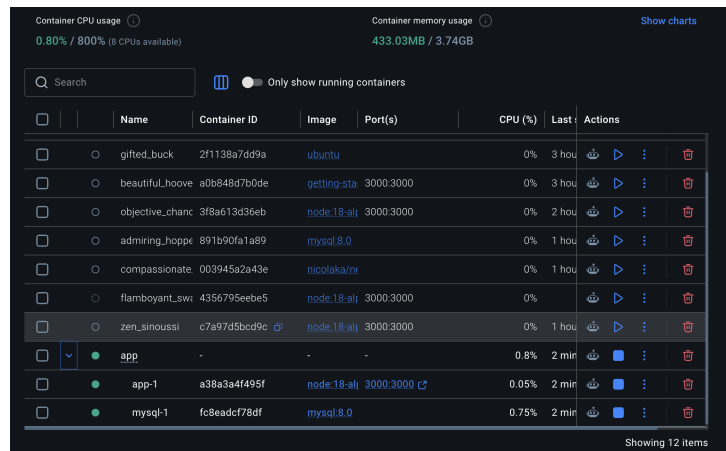


Figure 31: Application in the Docker Desktop UI

In Docker Desktop, we can see that the containers started by Docker Compose are grouped under one project. This makes it easier to manage the application and check details about running containers.

At the end, we used the following command to stop the containers:

```
docker compose down
```

3.9 Finishing the project

As a final step, we checked some good practices related to Docker images. First, we focused on security scanning. The tutorial mentions `docker scan`, but on our machine this command was deprecated and did not work anymore. For this reason, we used the newer Docker Scout command:

```
docker scout quickview getting-started
```

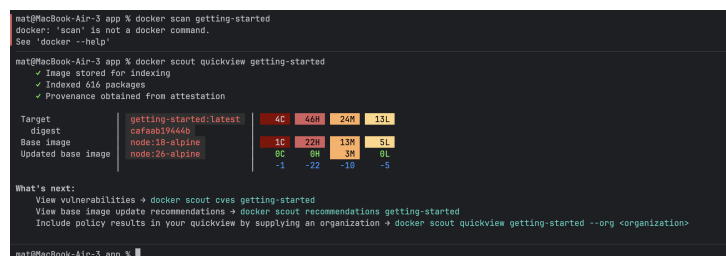


Figure 32: Security scanning the getting-started image

Next, we focused on image layers. Once a layer changes, all layers after it have to be recreated as well.

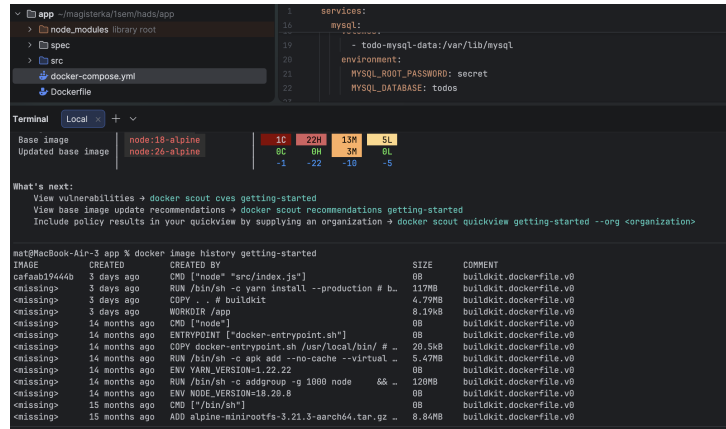


Figure 33: Docker image history command

Each line represents a layer in the image. The output shows the base layers at the bottom and the newest layer at the top. This is useful when diagnosing why an image is large or why a build takes more time.

Then, we moved to layer caching. To improve caching, we updated the Dockerfile in the following way:

```
FROM node:18-alpine
WORKDIR /app
COPY package.json yarn.lock ./
RUN yarn install --production
COPY . .
CMD ["node", "src/index.js"]
```

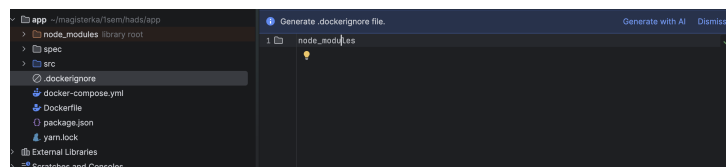


Figure 34: .dockerignore file

```
docker:desktop-linux
[+] Building 17.6s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring Dockerfile: 170B 0.0s
=> [internal] load metadata for docker.io/library/node:18-alpine 0.1s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 52B 0.0s
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:866421d663b6c28fd3ebc498332f249011d118945588d8a35cb9c48ca9d9e 0.0s
=> => resolve docker.io/library/node:18-alpine@sha256:866421d663b6c28fd3ebc498332f249011d118945588d8a35cb9c48ca9d9e 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 12.17kB 0.0s
=> CACHED [2/5] WORKDIR /app 0.0s
=> [3/5] COPY package.json yarn.lock ./ 0.1s
=> [4/5] RUN yarn install --production 12.8s
=> [5/5] COPY . 0.3s
=> => exporting to image 4.1s
=> => exporting layers 2.7s
=> => exporting manifest sha256:592ca777f5b59f8a52cabede85c564de9db8989f1f0038c78234af9b18b9 0.0s
=> => exporting config sha256:83a52a59d37f6e8d57c48184d837207c6f4d69f01c2b8a837090b9f 0.0s
=> => exporting attestation manifest sha256:e8073c143e0f8279b5f8b6ff431b6944d0d7b315c173275ffff51bdfc8a3db 0.0s
=> => exporting manifest list sha256:d777a7b69ed944a235c18d2835c8eb08678bb89284c6e03e78927ca51e2d2c34 0.0s
=> => naming to docker.io/library/getting-started:latest 0.0s
=> => unpacking to docker.io/library/getting-started:latest 1.3s
```

Figure 35: First Docker build

Then, we changed the title in the application, so the image had to be rebuilt.

```
docker:desktop-linux
[+] Building 0.4s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring Dockerfile: 170B 0.0s
=> [internal] load metadata for docker.io/library/node:18-alpine 0.0s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 52B 0.0s
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:866421d663b6c28fd3ebc498332f249011d118945588d8a35cb9c48ca9d9e 0.0s
=> => resolve docker.io/library/node:18-alpine@sha256:866421d663b6c28fd3ebc498332f249011d118945588d8a35cb9c48ca9d9e 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 3.93kB 0.0s
=> CACHED [2/5] WORKDIR /app 0.0s
=> CACHED [3/5] COPY package.json yarn.lock ./ 0.0s
=> CACHED [4/5] RUN yarn install --production 0.0s
=> [5/5] COPY . 0.0s
=> => exporting to image 0.2s
=> => exporting layers 0.1s
=> => exporting manifest sha256:3c1ade0d4f74673c3888d8c19e5b98e4f190b9d2295159b8d4f3f32899d4d 0.0s
=> => exporting config sha256:37d129c09c549a0f56a55a8c9711a968319a385ea50464e29c94c0b304da 0.0s
=> => exporting attestation manifest sha256:5a76958c8268db8cb0d403c117c0d7342a94fa5e7c0b200e11ba6a3a8854 0.0s
=> => exporting manifest list sha256:dff8a8d079c2b123a1a79432b24c6eef84d1120d1a8c7ef971c85315c7f 0.0s
=> => naming to docker.io/library/getting-started:latest 0.0s
=> => unpacking to docker.io/library/getting-started:latest 0.0s
```

Figure 36: Docker build re-run

Comparing the first and second build, we can see that during the second run some layers were cached. As a result, the build was faster because Docker did not have to repeat all steps from the beginning.

The following figure shows the last page of the successfully finished tutorial.

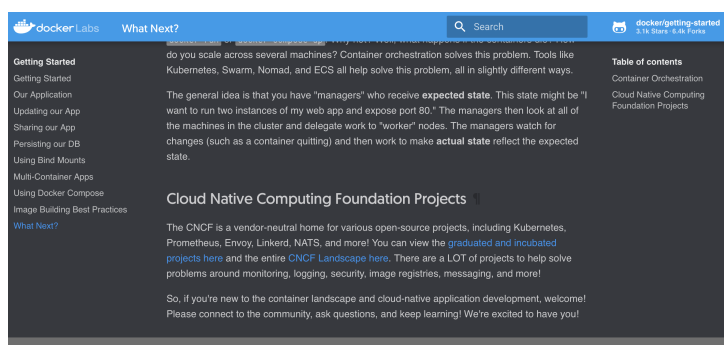


Figure 37: Successfully finished tutorial

Beyond the tutorial itself, we also performed one additional housekeeping step to make our containers easier to identify in Docker Desktop:


```
docker rename <old_name> <new_name>
```

```
mit@MacBook-Air-3 app % docker rename app-app-1 Chabraszewski_Fydnych_Jedrzecznyk_app
docker rename app-mysql-1 Chabraszewski_Fydnych_Jedrzecznyk_mysql
docker rename 0d1383bf8e2 Chabraszewski_Fydnych_Jedrzecznyk_getting-started

mit@MacBook-Air-3 app % docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
a38a3a4f495f   node:18-alpine                    "docker-entrypoint.s..." 14 hours ago   Up 14 hours   0.8.0.8:3000->3000/tcp, :::3000->3000/tcp
Chabraszewski_Fydnych_Jedrzecznyk_app
fc8eadcf78df   mysql:8.0                        "docker-entrypoint.s..." 14 hours ago   Up 14 hours   3306/tcp, 33060/tcp
Chabraszewski_Fydnych_Jedrzecznyk_mysql
c7a97d5bcd9c   node:18-alpine                    "docker-entrypoint.s..." 15 hours ago   Exited (1) 14 hours ago
zen_sinoussi
4356795eebe5   node:18-alpine                    "docker-entrypoint.s..." 15 hours ago   Created
flamboyant_swartz
```

Figure 38: Renamed containers

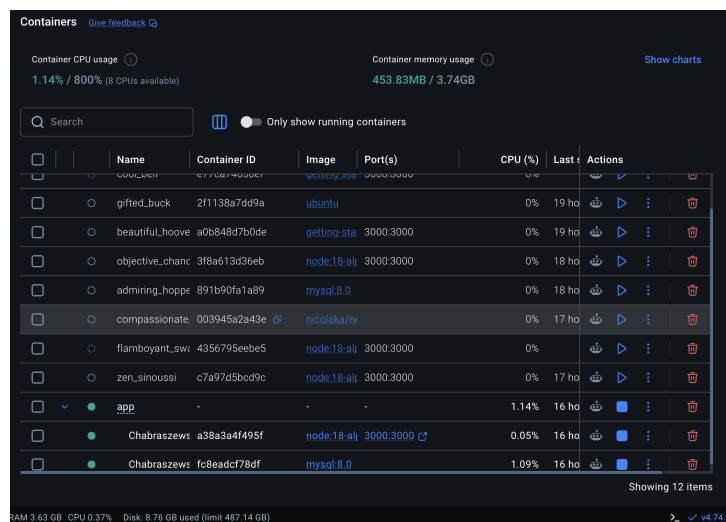


Figure 39: Docker Desktop containers view

4 Conclusions

During this task, we learned much more than only how to run a few Docker commands. At the beginning, Docker looked like a tool that simply starts applications in containers, but during the tutorial we understood that it also helps to organize the whole application environment. Thanks to Docker, the application can be started in a similar way on different computers, without installing all dependencies manually.

One of the most important things we learned was the difference between an image and a container. An image is like a prepared template of the application, while a container is a running instance created from this image. This difference became clearer when we built our own image from a Dockerfile and then used it to start the todo application.

Another important lesson was connected with data persistence. At first, we lost our todo items after removing the container. This showed us that data stored only inside a container is not safe if the container is deleted. Later, after using a Docker volume, the data was still available even after removing and starting the container again. This was one of the most practical parts of the task, because it showed a real problem that can happen in containerized applications.

We also practiced bind mounts. They were useful because we could change files locally and see the result in the running application without rebuilding the image every time. This made us understand why bind mounts are helpful during development, while volumes are more useful for storing application data.

The multi-container part was also very useful. We connected the todo application with a MySQL database running in a separate container. This helped us understand that real applications often need more than one container. We also saw that containers can communicate with each other through a Docker network and that Docker can use container names for communication between services.

Docker Compose was probably the most convenient part of the tutorial. Before using Compose, we had to remember many commands, create networks manually, set environment variables and start containers separately. With Docker Compose, all of this configuration was placed in one YAML file. This made the whole setup easier to run, stop, and share with another person.

At the end, we also looked at image layers, caching, and basic security scanning. We learned that the order of instructions in a Dockerfile can affect build time. When Docker can reuse cached layers, rebuilding the image is faster. We also noticed that some Docker commands from tutorials can

become outdated for example `docker scan`, so sometimes it is necessary to use newer tools such as Docker scout.

This task also improved our practical troubleshooting skills. We had to deal with problems such as a port already being used, containers that had to be stopped or removed, and commands that did not work exactly as expected. Solving these problems helped us understand Docker better than only reading about it.

Overall this task gave us a good introduction to Docker and containerized applications. We learned about Docker Desktop, Dockerfiles, images, containers, port mapping, Docker Hub, volumes, bind mounts, networks, MySQL in a container, Docker Compose and basic image optimization. We still need more practice to use Docker in more advanced or production like projects, but after this task we understand the main ideas much better.